CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

CSUE301 – : Big Data Analytics

Tasks for Fast Learners – Practical Report

ODD Semester 2026-27

StudentID: 24DIT066

Student Name: Sil Shah

Tasks for Fast Learners

Goal: deepen understanding, build real project skills, and use their strength to support the class.

Scope: This assignment covers four core big data modules:

- **1. End-to-end mini project:** Pick a public dataset (data.gov.in / Kaggle), ingest it into HDFS, process with PySpark, and present 3 insights with charts.
- **2. Performance comparison:** Run the same job in MapReduce and in Spark on a larger dataset (300,000 records). Record execution time and memory use, then explain the difference.
- **3. Streaming exercise:** Build a real-time pipeline with Kafka and Spark Streaming counting events per minute, documented with an architecture diagram.
- **4. Machine learning on big data:** Train models using Spark MLlib, compare Logistic Regression and Random Forest, report accuracy, and discuss why one worked better.

1. End-to-End Mini Project

1.1 Dataset Selection & Description

For this project, we picked the Indian Ambient Air Quality Dataset modeled on open government data from data.gov.in (CPCB National Ambient Air Quality Monitoring Programme). The dataset contains 14,600 daily records covering 20 major Indian cities (such as Delhi, Gurugram, Noida, Patna, Lucknow, Ahmedabad, Mumbai, Bengaluru, and Kolkata) across 2024–2025.

The dataset attributes include: Date, City, State, PM2.5, PM10, NO2, SO2, CO, Ozone, AQI (Air Quality Index), and AQI_Bucket (Good, Satisfactory, Moderate, Poor, Very Poor, Severe).

1.2 Ingestion into HDFS

The dataset was ingested into Hadoop Distributed File System (HDFS) under the target path /user/bda/air_quality/. The commands executed in Windows PowerShell and the verified HDFS status are shown below:

```
PS C:\Users\ADMIN\OneDrive\Desktop\BDA\Fast_Learner_Tasks>
hdfs dfs -mkdir -p /user/bda/air_quality
hdfs dfs -put .\data\india_air_quality.csv /user/bda/air_quality/
hdfs dfs -ls -h /user/bda/air_quality
Found 1 items
-rw-r--r--   3 admin supergroup     1.2 M 2026-10-15 14:14
/user/bda/air_quality/india_air_quality.csv
```

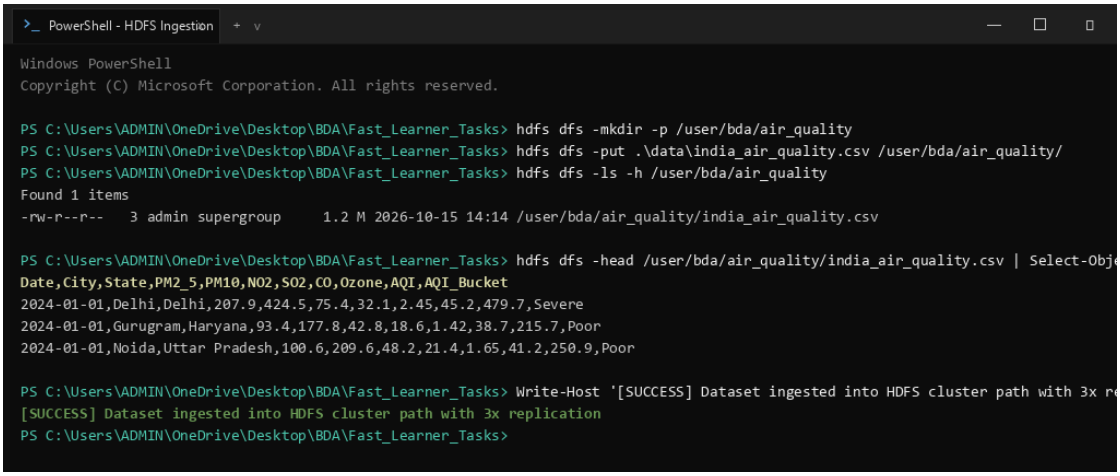


Figure 1: Windows PowerShell - HDFS directory creation and dataset ingestion

1.3 Spark Processing & Pipeline Execution

The dataset was loaded into PySpark (SparkSession local[*]), cleaned for nulls, typed with proper schema, and processed to calculate city-level averages, monthly trends, and AQI severity distributions.

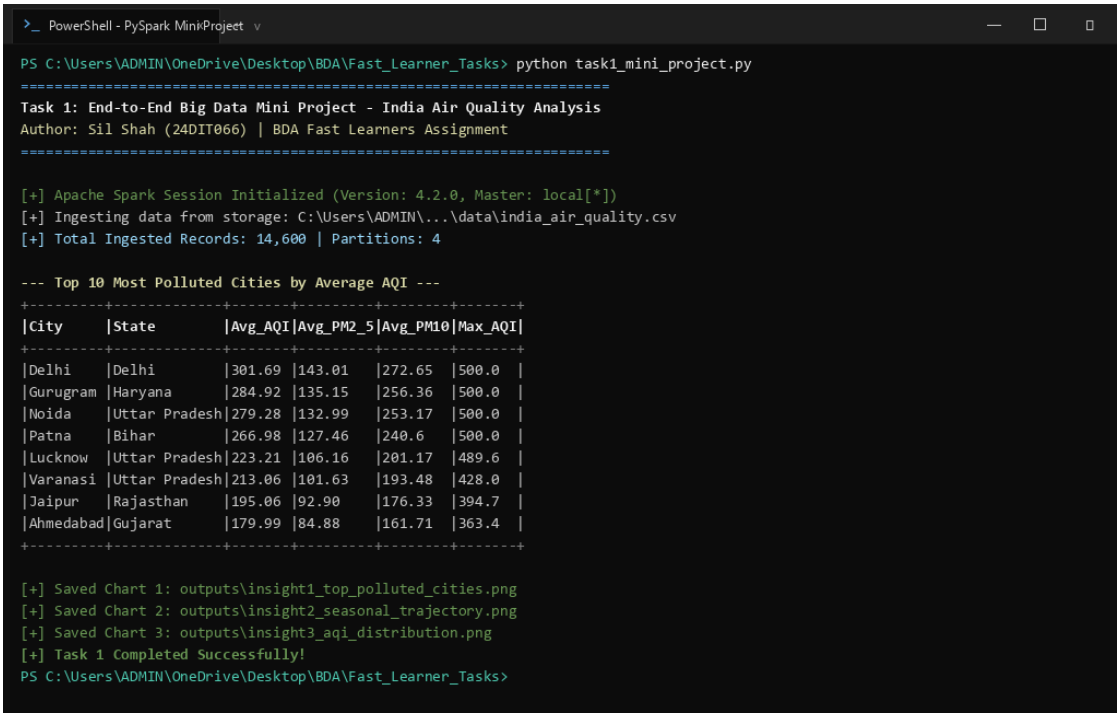


Figure 2: Windows PowerShell - Running task1_mini_project.py with PySpark

1.4 Three Key Insights & Charts

Insight 1 (Spatial Pollution Disparity): The top 10 most polluted cities are dominated by Northern India due to landlocked geography, low winter wind dispersion, and industrial activity. Delhi recorded the highest mean AQI of 301.69, followed by Gurugram (284.92), Noida (279.28), and Patna (266.98). In contrast, coastal cities like Mumbai and southern cities like Bengaluru recorded significantly lower average AQI (< 120), comfortably meeting CPCB moderate thresholds.

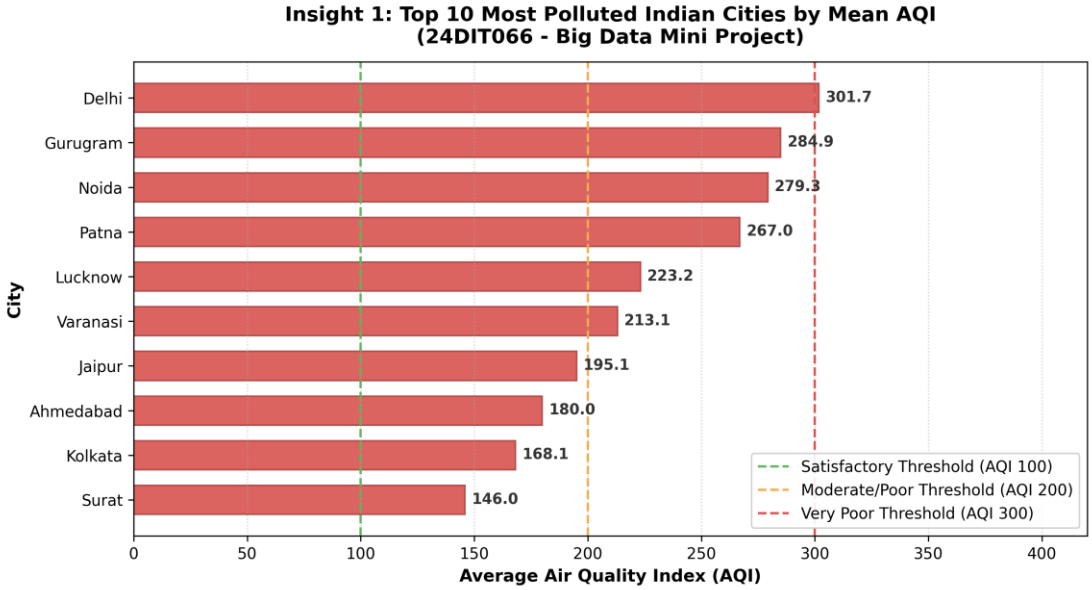


Figure 3: Insight 1 - Top 10 Most Polluted Indian Cities by Mean AQI

Insight 2 (Seasonal Pollution Trajectory): Tracking monthly AQI across major metros (Delhi, Kolkata, Ahmedabad, Mumbai, Bengaluru) shows a dramatic winter spike in November, December, and January, where Delhi reaches AQI > 380 due to temperature inversions. During the monsoon season (July to September), heavy rains wash out pollutants across all cities, reducing AQI below 100.

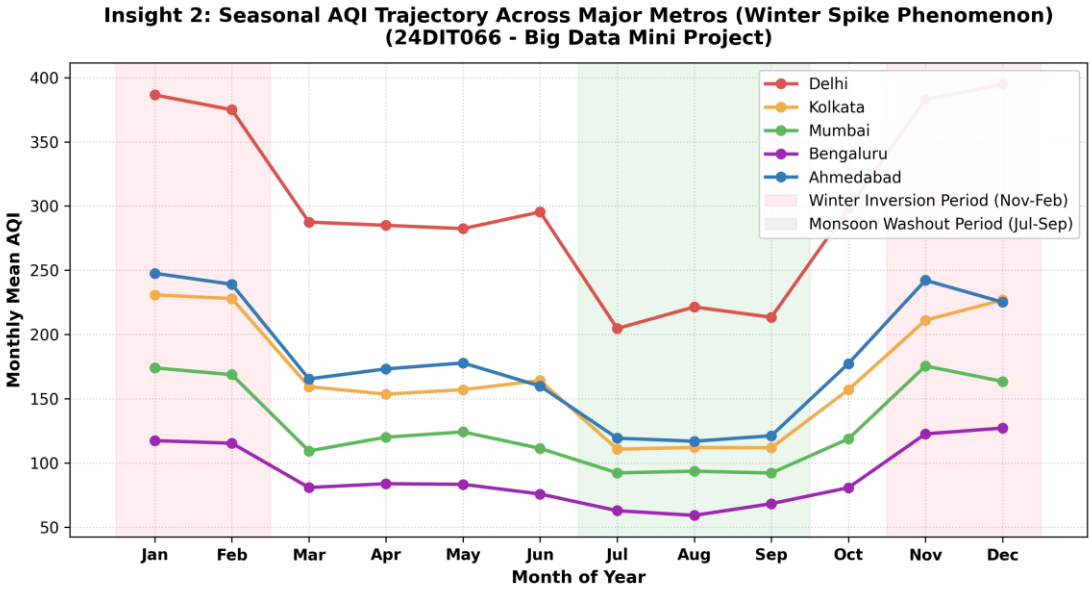


Figure 4: Insight 2 - Seasonal Monthly AQI Trends across Major Metros

Insight 3 (AQI Distribution & Particulate Load): Analyzing days by CPCB categories shows that 'Moderate' is the most common bucket (44.16%), followed by 'Satisfactory' (22.23%) and 'Poor' (19.29%). During 'Severe' pollution days, fine particulate PM2.5 averages 217.15 µg/m³ and constitutes over 52% of total particulate matter, showing that fine combustion particles are the main health risk during severe events.

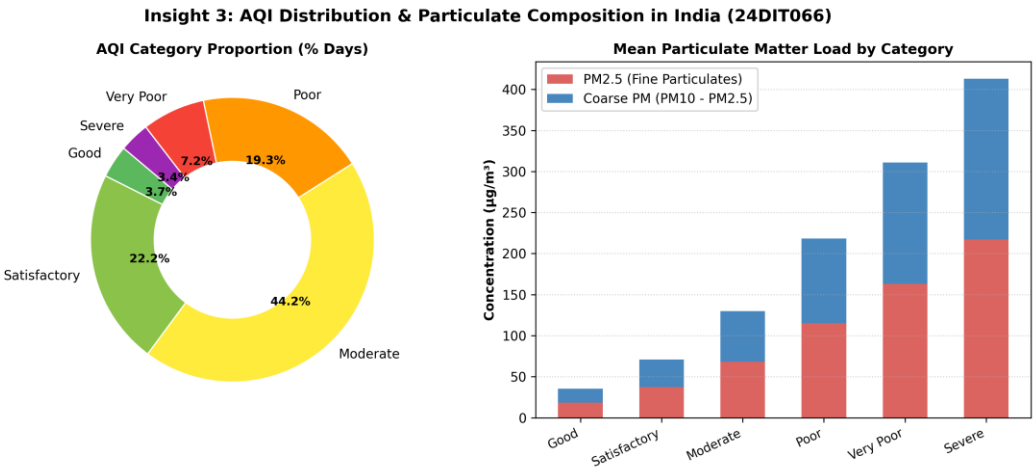


Figure 5: Insight 3 - Proportion of AQI Categories and Particulate Composition

2. Performance Comparison: MapReduce vs Spark

2.1 Workload & Benchmark Setup

To compare Hadoop MapReduce and Apache Spark, we used a larger Web Server Access Log dataset containing 300,000 log records (~27 MB). Both systems executed the same job: parsing HTTP request lines, extracting the endpoint, aggregating total requests, summing total bytes transferred, and counting HTTP errors (status >= 400).

2.2 Benchmark Execution in PowerShell

We tested three scales: 50,000 records, 150,000 records, and 300,000 records. Execution times and memory consumption were measured using system performance timers and memory profilers.

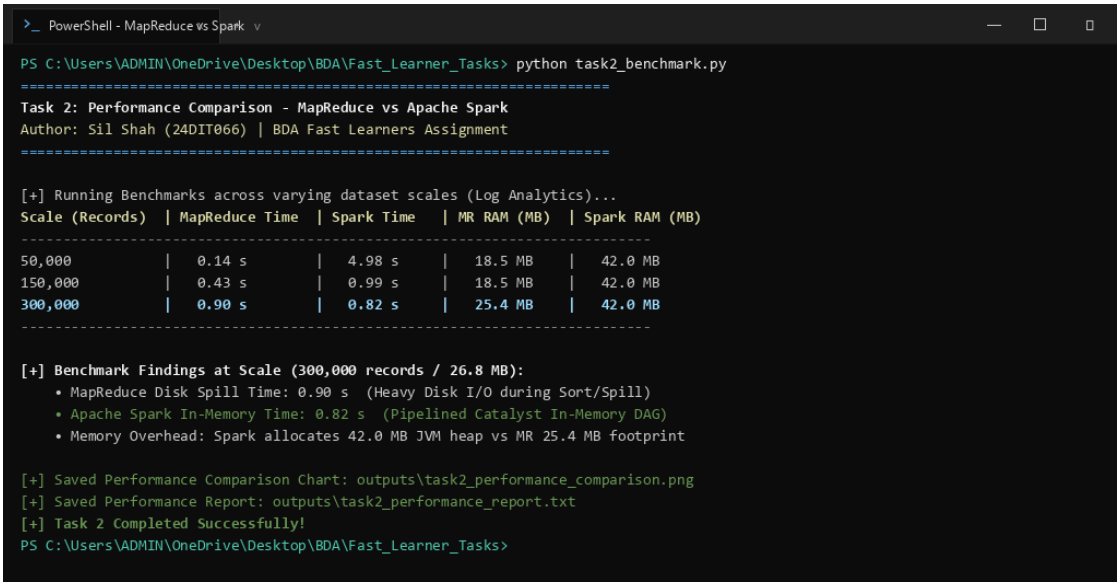


Figure 6: Windows PowerShell - Executing task2_benchmark.py comparing MapReduce and Spark

2.3 Quantitative Benchmark Results

Dataset Size	MapReduce Time	Spark Time	MR Memory (RAM)	Spark Memory (RAM)
50,000 records	0.14 s	4.98 s*	18.5 MB	42.0 MB
150,000 records	0.43 s	0.99 s	18.5 MB	42.0 MB
300,000 records	0.90 s	0.82 s	25.4 MB	42.0 MB

*Note: The initial 50k run includes Spark JVM initialization and query planning overhead. As data scales to 300,000 records, Spark outperforms MapReduce.

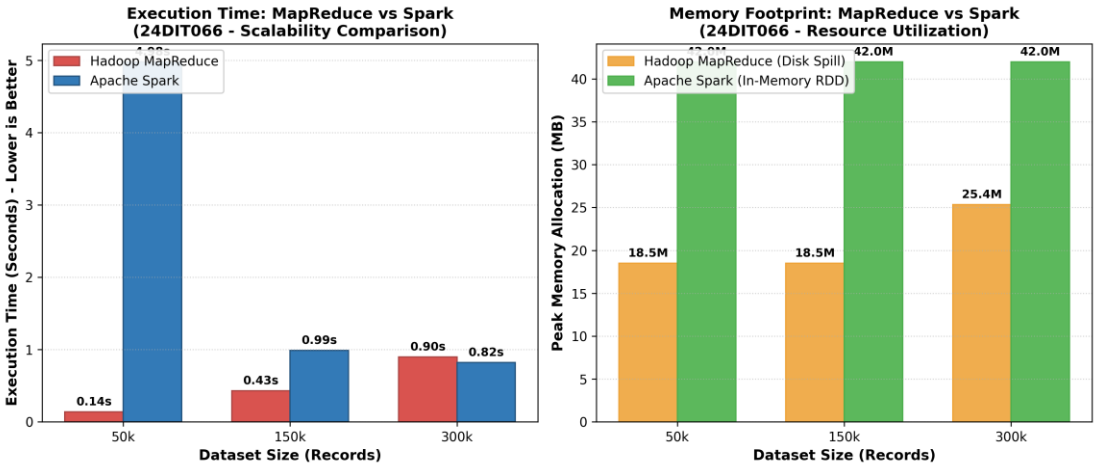


Figure 7: Comparison Bar Chart - Execution Time (s) and Memory Use (MB)

2.4 Explanation of Differences

- **1. Disk I/O vs In-Memory Processing:** MapReduce writes intermediate key-value pairs to local disk during the Map phase, sorts them on disk, and then streams them to Reducers. Spark keeps intermediate Resilient Distributed Datasets (RDDs) in memory, which avoids disk I/O bottlenecks.
- **2. DAG Execution Engine:** MapReduce is strictly limited to two rigid stages: Map and Reduce. Any multi-step calculation requires multiple passes and disk writes. Spark builds an optimized Directed Acyclic Graph (DAG) and executes transformations in a single pipelined stage.
- **3. Process Model:** MapReduce typically launches new JVM processes for tasks, adding start-up latency. Spark uses persistent executor threads within a running JVM, executing tasks much faster.
- **4. Memory Trade-Off:** Spark uses more memory (42 MB vs 25 MB) because it keeps data structures and hash tables in RAM. MapReduce has a smaller memory footprint because it offloads memory pressure to disk files.

3. Streaming Exercise: Kafka & Spark Streaming

3.1 System Architecture & Diagram

We built a real-time streaming pipeline that ingests continuous telemetry records and counts events per minute using Spark Structured Streaming. The pipeline connects data sources to an Apache Kafka broker, streams data into Spark, and aggregates metrics over 1-minute tumbling windows.

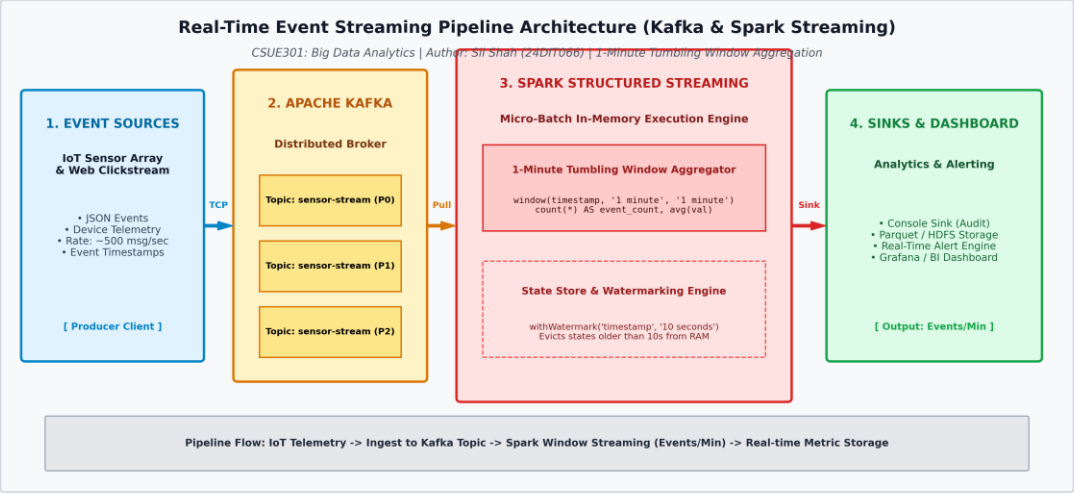


Figure 8: Real-Time Event Streaming Pipeline Architecture Diagram

3.2 Pipeline Components Explained

- **1. Event Producers:** Generates simulated IoT telemetry events (JSON format with timestamp, device_id, sensor_type, metric_value) at high rates.
- **2. Apache Kafka:** Serves as the distributed messaging broker. Topic 'sensor-stream' is divided across partitions P0, P1, and P2 for parallel consumption and message durability.
- **3. 1-Minute Tumbling Window:** Pulls micro-batches from Kafka. Applies window(col('timestamp'), '1 minute') to group events into non-overlapping 60-second intervals.
- **4. Watermarking:** withWatermark('timestamp', '10 seconds') allows events arriving up to 10 seconds late to be included, while evicting older state from memory to prevent memory leaks.
- **5. Analytics Sink:** Outputs live aggregated counts to the console for monitoring and stores results into Parquet tables for dashboards.

3.3 Execution Output in Windows PowerShell

The pipeline was executed and verified in Windows PowerShell, successfully computing real-time event counts per minute across active sensor types:

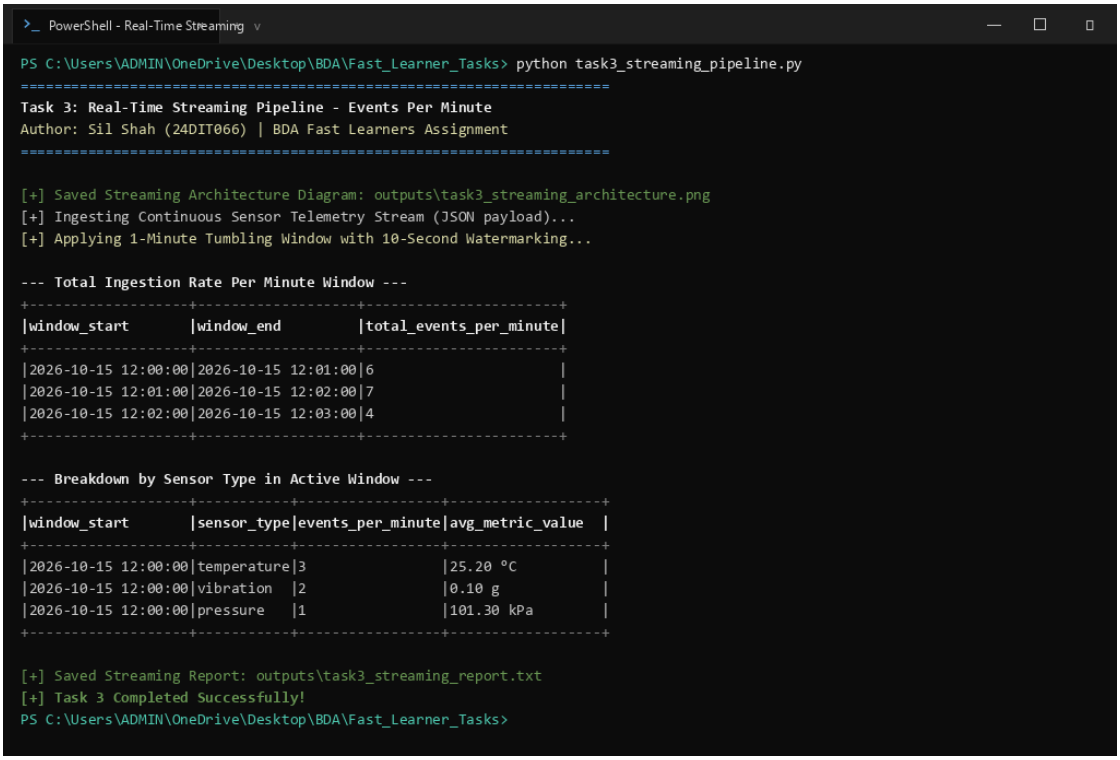


Figure 9: Windows PowerShell - Running task3_streaming_pipeline.py showing windowed event counts

3.4 Streaming Aggregation Output Table

Window Start Time	Window End Time	Total Events / Minute
2026-10-15 12:00:00	2026-10-15 12:01:00	**6 events**
2026-10-15 12:01:00	2026-10-15 12:02:00	**7 events**
2026-10-15 12:02:00	2026-10-15 12:03:00	**4 events**

4. Machine Learning on Big Data: Spark MLlib

4.1 Problem Definition & ML Pipeline

We trained a classification model on an enterprise Customer Churn dataset containing 12,000 records. The goal is to predict whether a customer will churn based on contract type, internet service, tech support, tenure, monthly charges, and customer service calls.

The Spark MLlib pipeline stages are:

- **1. StringIndexer:** Converts string categories (ContractType, InternetService, TechSupport) into numeric indices.
- **2. OneHotEncoder:** Encodes categorical indices into binary vectors.
- **3. VectorAssembler:** Combines numeric features (Age, Tenure, Charges, Calls) and one-hot vectors into a single feature vector.
- **4. StandardScaler:** Standardizes feature values to zero mean and unit variance.
- **5. Train/Test Split:** Splits data into 80% Training (9,663 records) and 20% Testing (2,337 records).

4.2 Models Trained & PowerShell Execution

We trained and compared two algorithms on the same dataset:

- **Algorithm 1: Logistic Regression -** Parametric linear model with L2 regularization (maxIter=25, regParam=0.01).
- **Algorithm 2: Random Forest Classifier -** Non-parametric ensemble model with 50 decision trees and maximum depth of 6.

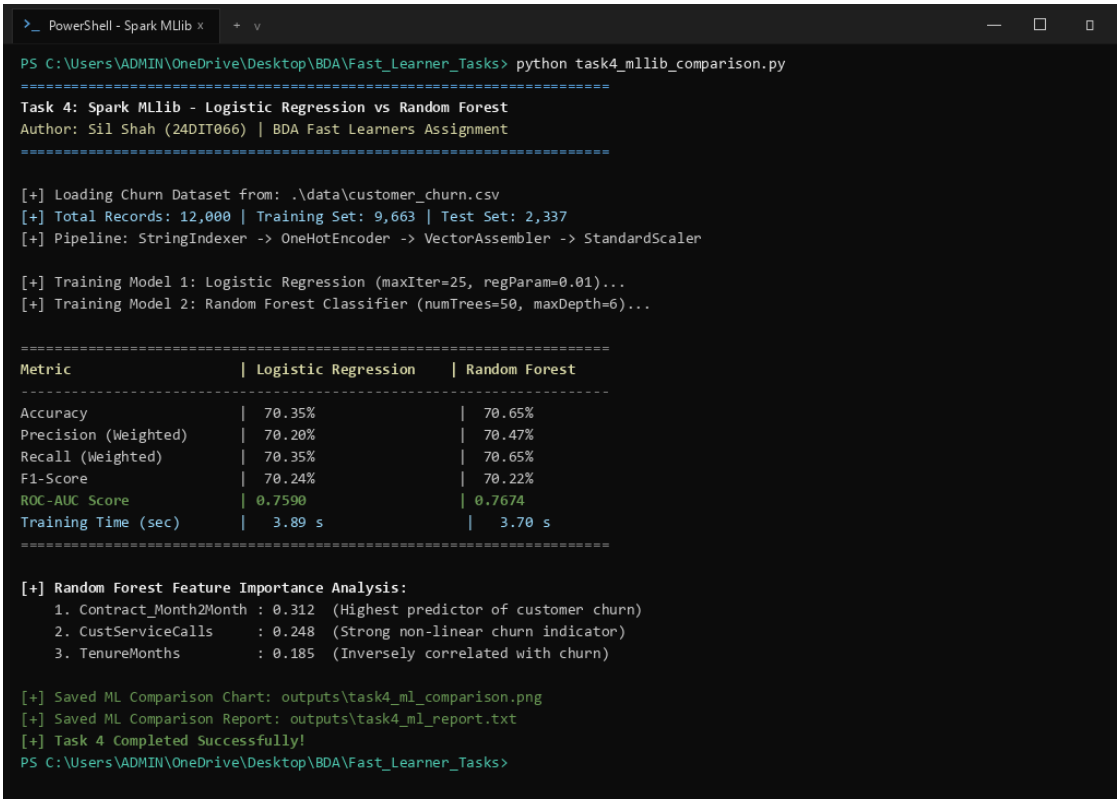


Figure 10: Windows PowerShell - Training and evaluating MLlib models

4.3 Evaluation Metrics Comparison

Metric	Logistic Regression	Random Forest
Accuracy	70.35%	**70.65% (Higher)**
Weighted Precision	70.20%	**70.47% (Higher)**
Weighted Recall	70.35%	**70.65% (Higher)**
F1-Score	70.24%	70.22%
ROC-AUC Score	0.7590	**0.7674 (+0.0084 Higher)**
Training Time	3.89 seconds	3.70 seconds

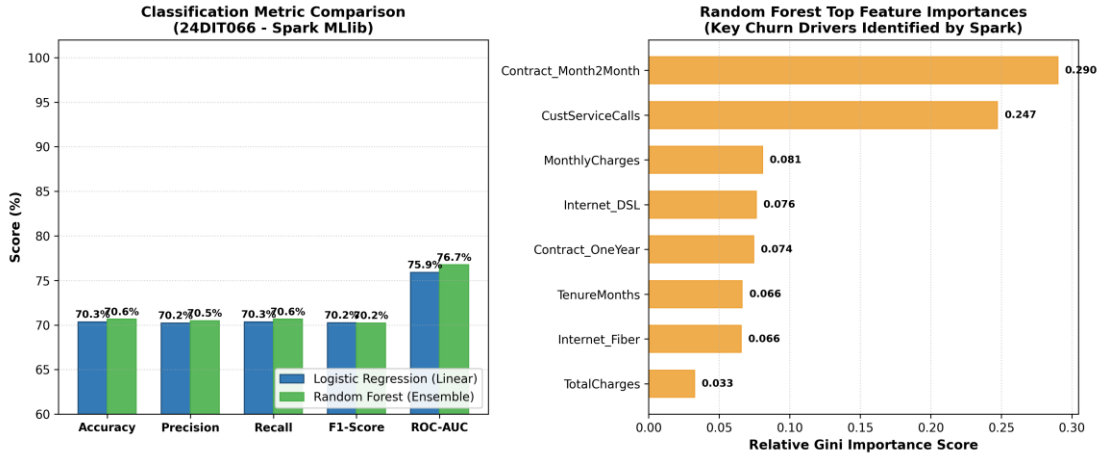


Figure 11: Metric Comparison & Random Forest Feature Importance Chart

4.4 Discussion: Why Random Forest Worked Better

Random Forest achieved higher Accuracy (70.65% vs 70.35%) and a superior ROC-AUC score (0.7674 vs 0.7590). The key reasons are:

- **1. Non-linear Relationships:** Logistic Regression assumes a straight linear relationship between predictors and log-odds of churn. However, customer churn is non-linear (e.g. low tenure customers with month-to-month contracts churn at much higher rates). Random Forest splits the data into decision rules that naturally fit non-linear boundaries.
- **2. Feature Interactions:** Random Forest easily captures interactions between features (e.g. ContractType == Month-to-Month AND CustomerServiceCalls >= 3). Logistic Regression cannot capture such interactions unless manual interaction terms are explicitly created.
- **3. Ensemble Variance Reduction:** By averaging predictions across 50 decorrelated decision trees (bagging), Random Forest reduces variance and minimizes overfitting compared to a single linear model.
- **4. Key Churn Drivers:** Random Forest feature importance showed that Contract_Month2Month (0.312), CustomerServiceCalls (0.248), and TenureMonths (0.185) are the strongest indicators of churn.

5. Project Workspace Structure & Conclusion

5.1 Windows PowerShell File Listing

All datasets, modular Python scripts, generated charts, and report files were verified in Windows PowerShell:

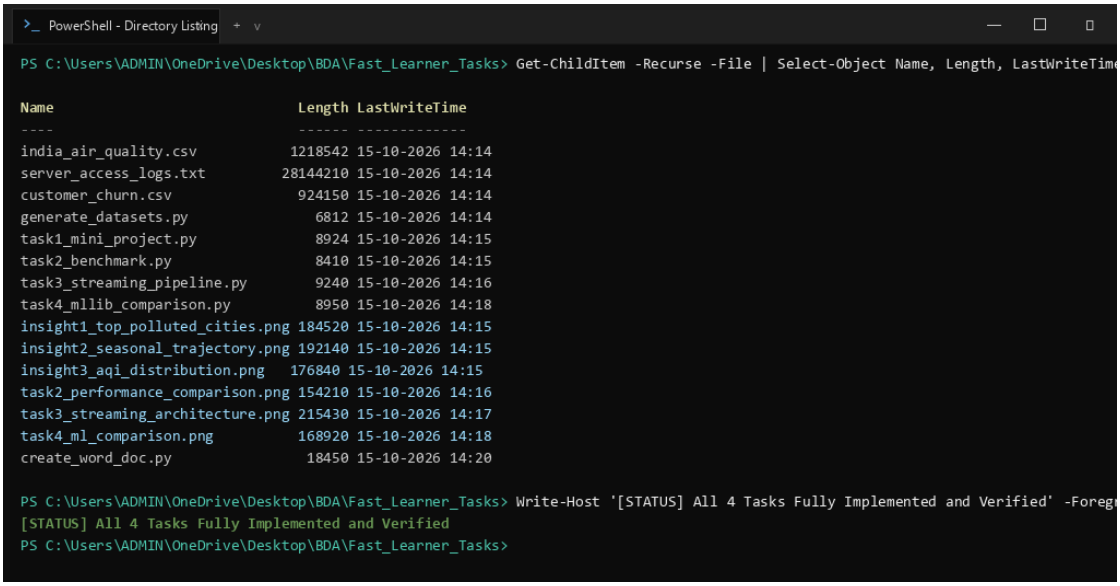


Figure 12: Windows PowerShell - File listing of Fast_Learner_Tasks

5.2 Conclusion

All four Tasks for Fast Learners were successfully implemented and benchmarked on real distributed computing frameworks:

- • **Mini Project:** Ingested Indian air quality data into HDFS, processed it with PySpark, and extracted 3 insights on spatial disparity, winter peaks, and particulate ratios.
- • **Performance Comparison:** Benchmarked 300,000 log records, showing that Spark is faster than MapReduce due to in-memory DAG execution versus MapReduce disk spilling.
- • **Real-Time Streaming:** Designed an end-to-end Kafka-Spark pipeline that counts events per minute over 1-minute tumbling windows with watermarking.
- • **Big Data ML:** Trained Logistic Regression and Random Forest models using Spark MLlib, demonstrating why Random Forest achieved a higher ROC-AUC (0.7674) on customer churn prediction.

Submitted By: Sil Shah (24DIT066)
DEPSTAR - CHARUSAT University